Apache Spark
Complete Learning Guide

Basic to Medium | PySpark & Spark SQL | With Code Examples & Diagrams

A structured guide from zero to competent Spark developer

* Spark Architecture & Internals
* RDDs, DataFrames, Spark SQL
* Joins, Window Functions, Aggregations
* Reading & Writing All File Formats
* Performance Tuning & Optimization
* Structured Streaming Basics
* Real-World ETL Patterns
* Quick Reference Cheat Sheets

Table of Contents

PART 1: SPARK FOUNDATIONS
Chapter 1: What is Apache Spark?
Chapter 2: Spark Architecture
Chapter 3: Setting Up Spark
Chapter 4: RDD Fundamentals
Chapter 5: Partitions & Parallelism
PART 2: SPARK SQL & DATAFRAMES
Chapter 6: DataFrame Basics
Chapter 7: DataFrame Transformations
Chapter 8: Aggregations & GroupBy
Chapter 9: Joins
Chapter 10: Window Functions
Chapter 11: Working with Different Data Types
Chapter 12: Spark SQL
PART 3: READING & WRITING DATA
Chapter 13: File Formats
Chapter 14: Reading Data
Chapter 15: Writing Data
Chapter 16: Delta Lake Integration
PART 4: PERFORMANCE & OPTIMIZATION
Chapter 17: Understanding the Spark UI
Chapter 18: Common Performance Problems & Fixes
Chapter 19: Caching & Persistence
Chapter 20: Broadcast Variables & Accumulators
Chapter 21: Configuration Tuning
PART 5: REAL-WORLD PATTERNS
Chapter 22: ETL Pipeline Pattern
Chapter 23: Incremental Processing
Chapter 24: Structured Streaming Basics
Chapter 25: Testing Spark Applications
QUICK REFERENCE
PySpark Cheat Sheet
Spark SQL Functions Cheat Sheet
Configuration Cheat Sheet

PART 1: SPARK FOUNDATIONS

Chapter 1: What is Apache Spark?

Apache Spark is a unified analytics engine for large-scale data processing. It was developed at UC Berkeley in 2009 as a response to the limitations of Hadoop MapReduce, and later donated to the Apache Software Foundation.

1.1 Why Spark Was Created - MapReduce Limitations

Hadoop MapReduce was the original big data processing framework. However, it had critical limitations that made it unsuitable for modern data workloads:

- Disk-based: MapReduce writes intermediate results to disk between map and reduce stages, making it extremely slow for iterative algorithms (like ML)
- Complex API: Writing even simple operations required verbose Java code with Mapper/Reducer classes
- No real-time: MapReduce is purely batch - no support for streaming or interactive queries
- Two-stage only: Everything must fit into a map phase and a reduce phase, limiting expressiveness
- Slow iteration: Each MapReduce job reads from and writes to HDFS. Iterative algorithms (run same computation multiple times) are painfully slow

1.2 Spark vs MapReduce Comparison

Feature	Hadoop MapReduce	Apache Spark
Processing Speed	Slow (disk-based I/O)	Up to 100x faster (in-memory)
Ease of Use	Verbose Java code	Simple Python/Scala/SQL API
Real-time Processing	No (batch only)	Yes (Spark Streaming)
Iterative Algorithms	Very slow (disk I/O each iteration)	Fast (caches data in memory)
Fault Tolerance	Replication-based	RDD lineage (recompute from DAG)
Interactive Queries	No	Yes (Spark SQL, notebooks)
Languages	Java	Python, Scala, Java, R, SQL
Community / Adoption	Declining	Dominant big data engine

1.3 Spark Ecosystem Components

Spark is not a single tool - it is an ecosystem of tightly integrated libraries built on top of Spark Core:

- Spark Core: The foundation. Distributed task scheduling, memory management, fault recovery, storage interaction. Provides the RDD API.
- Spark SQL: Structured data processing with DataFrame/Dataset API and SQL queries. The most-used Spark module in data engineering.
- Spark Streaming / Structured Streaming: Real-time stream processing. Structured Streaming (newer) treats streams as unbounded tables.
- MLlib: Machine learning library with algorithms for classification, regression, clustering, collaborative filtering, etc.
- GraphX: Graph processing engine for social network analysis, page ranking, etc. Less commonly used in data engineering.

1.4 Where Spark Runs

Platform	Type	Notes
Local Mode	Development	Single machine, for testing and learning
Standalone	Self-hosted cluster	Spark built-in cluster manager
YARN	Hadoop ecosystem	Most common in on-prem Hadoop clusters
Kubernetes	Container orchestration	Growing adoption, cloud-native
Databricks	Managed cloud service	Most popular managed Spark platform
AWS EMR	AWS managed	Elastic MapReduce with Spark
GCP Dataproc	GCP managed	Google Cloud managed Spark
Azure HDInsight	Azure managed	Microsoft managed Spark service

1.5 Spark Use Cases in Data Engineering

- ETL Pipelines: Extract data from sources, transform, load into data warehouse/lakehouse
- Data Lake Processing: Read/write Parquet, Delta, Iceberg tables at scale
- Data Quality: Validate data, detect anomalies, enforce schema
- Aggregation & Reporting: Pre-compute aggregates for dashboards
- Log Processing: Parse and analyze application/web server logs
- CDC (Change Data Capture): Process incremental changes with Delta Lake MERGE
- Real-time Analytics: Process streaming data from Kafka for near-real-time insights

Chapter 2: Spark Architecture

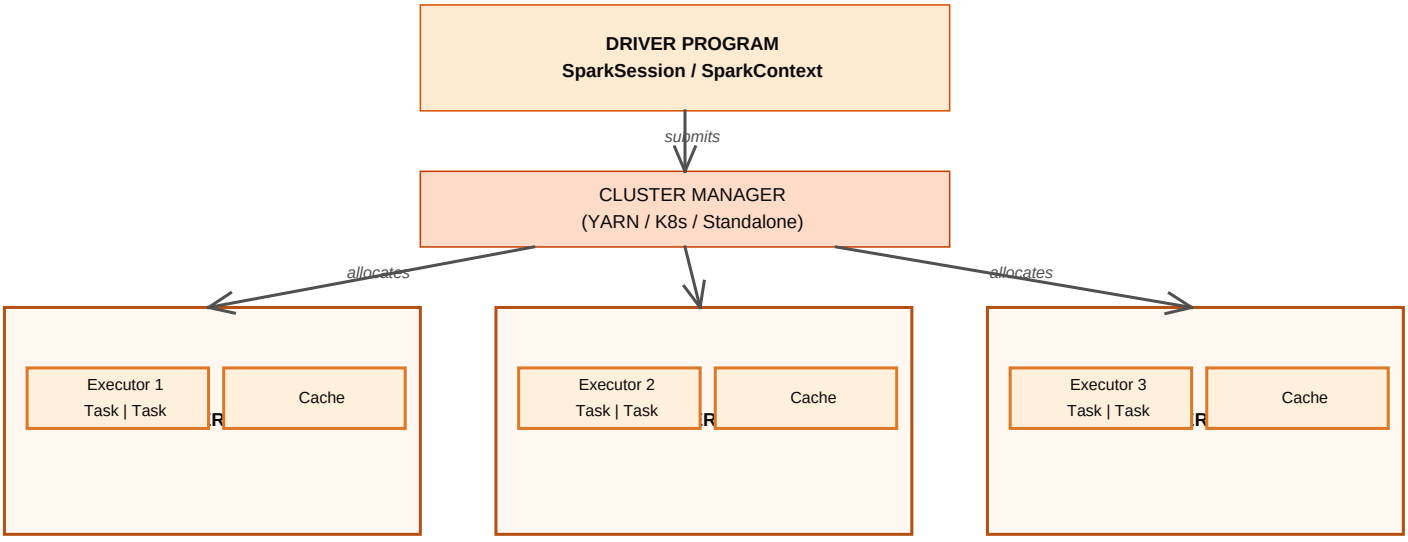
Understanding Spark architecture is essential for writing efficient applications and debugging performance issues.

2.1 Core Components

Spark uses a master-worker architecture with these key components:

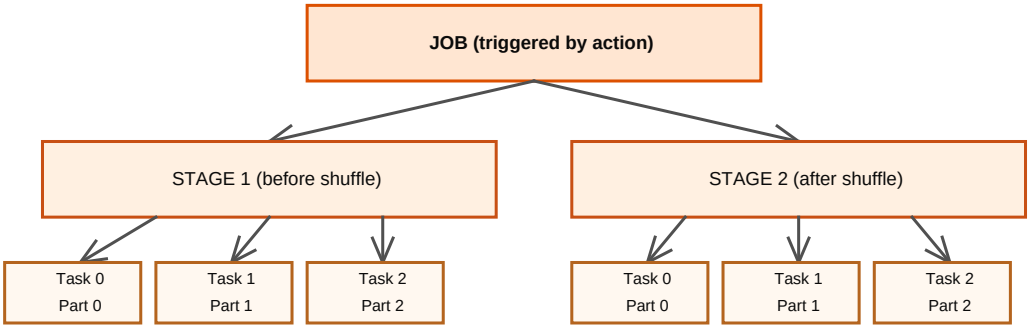
- Driver Program: The process that runs your main() function. Creates SparkSession, builds the DAG, and coordinates execution.
- SparkSession / SparkContext: The entry point to Spark functionality. SparkSession (Spark 2.0+) wraps SparkContext and provides unified API.
- Cluster Manager: External service that allocates resources across the cluster (YARN, K8s, Standalone, or Mesos).
- Worker Nodes: Machines in the cluster that run executors.
- Executors: JVM processes on worker nodes that run tasks and cache data. Each executor has allocated CPU cores and memory.
- Tasks: The smallest unit of work. One task processes one partition of data.

2.2 Architecture Diagram



2.3 Job --> Stage --> Task Hierarchy

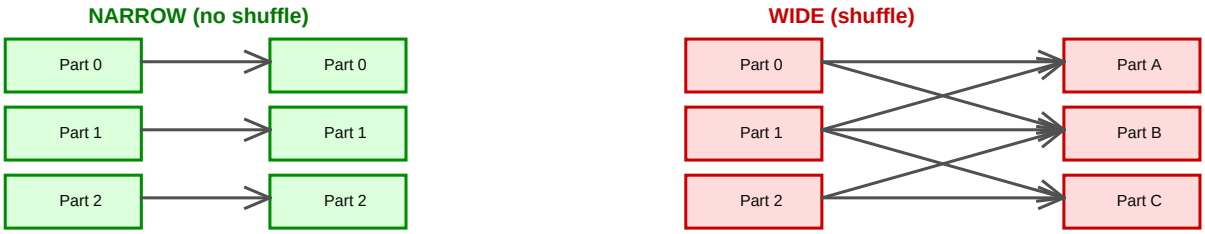
When you call an action (like collect(), count(), write()), Spark creates a Job. Each job is divided into Stages at shuffle boundaries. Each stage contains multiple Tasks (one per partition).



2.4 Narrow vs Wide Transformations

This distinction is critical for understanding Spark performance:

- Narrow transformations: Each input partition contributes to at most one output partition. Examples: map(), filter(), select(), withColumn(). NO shuffle required - fast!
- Wide transformations: Each input partition may contribute to multiple output partitions. Examples: groupBy(), join(), repartition(), orderBy(). Requires SHUFFLE - expensive!



2.5 Shuffle Explained

A shuffle is the most expensive operation in Spark. It occurs whenever data needs to be redistributed across partitions (e.g., groupBy, join, repartition). During a shuffle:

- Map side: Each task writes its output to local disk, sorted by target partition
- Shuffle service: Data is transferred across the network from map tasks to reduce tasks
- Reduce side: Each reduce task fetches its data from all map tasks, merges, and processes
- Shuffle writes data to disk and transfers it over the network - this is why it is slow
- The number of output partitions is controlled by spark.sql.shuffle.partitions (default: 200)

2.6 DAG (Directed Acyclic Graph)

When you chain transformations, Spark builds a DAG - a graph of computation steps. The DAG Scheduler optimizes this graph by:

- Combining narrow transformations into a single stage (pipelining)
- Creating stage boundaries at shuffle points
- Determining the most efficient execution plan
- Skipping unnecessary computations through lazy evaluation

You can view the DAG in the Spark UI (Jobs tab --> click a job --> DAG Visualization). This is essential for debugging performance.

Chapter 3: Setting Up Spark

3.1 Local Mode Setup

```
# Install PySpark (includes Spark engine)
pip install pyspark

# Verify installation
python3 -c "import pyspark; print(pyspark.__version__)"

# Optional: install with specific extras
pip install pyspark[sql]     # Spark SQL support
pip install pyspark[connect] # Spark Connect (remote)
```

3.2 SparkSession Creation

```
from pyspark.sql import SparkSession

# Basic SparkSession
spark = SparkSession.builder \
    .appName("MySparkApp") \
    .master("local[*]") \
    .getOrCreate()

# SparkSession with common configs
spark = SparkSession.builder \
    .appName("ProductionETL") \
    .master("yarn") \
    .config("spark.executor.memory", "8g") \
    .config("spark.executor.cores", "4") \
    .config("spark.executor.instances", "10") \
    .config("spark.driver.memory", "4g") \
    .config("spark.sql.shuffle.partitions", "200") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.serializer", "org.apache.spark.serializer.KryoSerializer") \
    .config("spark.sql.parquet.compression.codec", "snappy") \
    .enableHiveSupport() \
    .getOrCreate()

# Access underlying SparkContext
sc = spark.sparkContext
print(f"App Name: {spark.sparkContext.appName}")
print(f"Master: {spark.sparkContext.master}")
print(f"Spark Version: {spark.version}")
```

3.3 spark-submit Command

```
# Basic spark-submit
spark-submit my_job.py

# Full spark-submit with all common flags
spark-submit \
    --master yarn \
    --deploy-mode cluster \
    --name "MyETLJob" \
    --num-executors 10 \
    --executor-memory 8g \
    --executor-cores 4 \
    --driver-memory 4g \
    --conf spark.sql.shuffle.partitions=200 \
    --conf spark.sql.adaptive.enabled=true \
    --conf spark.dynamicAllocation.enabled=true \
    --conf spark.dynamicAllocation.minExecutors=5 \
    --conf spark.dynamicAllocation.maxExecutors=50 \
    --py-files utils.py,helpers.py \
    --jars extra-library.jar \
    --packages io.delta:delta-core_2.12:2.4.0 \
    my_job.py --date 2026-07-07

# deploy-mode options:
#   client  - Driver runs on submit machine (for debugging)
#   cluster - Driver runs on cluster (for production)
```

3.4 Spark UI Overview

When Spark runs, it starts a web UI (default: http://localhost:4040) with these tabs:

- Jobs tab: Lists all jobs. Each action (count, show, write) creates one job. Shows status, duration, stages.
- Stages tab: Shows stages within each job. Key metric: shuffle read/write bytes. Look for skewed tasks.
- Storage tab: Shows cached/persisted DataFrames with memory and disk usage.
- Environment tab: All Spark configs, system properties, classpath. Useful for debugging config issues.
- SQL tab: Shows DataFrame/SQL query plans (logical and physical). Essential for understanding query optimization.
- Executors tab: Shows executor resource usage - memory, disk, task counts. Helps identify resource bottlenecks.

Chapter 4: RDD Fundamentals

RDDs (Resilient Distributed Datasets) are the foundational data structure in Spark. While modern Spark applications should use DataFrames, understanding RDDs is important for interviews and for understanding how Spark works internally.

4.1 What is an RDD?

- Resilient: Can be recomputed from lineage if a partition is lost
- Distributed: Data is split across multiple nodes in the cluster
- Dataset: A collection of elements (rows, objects, tuples)
- Immutable: Once created, an RDD cannot be changed - you create new RDDs from existing ones
- Lazily evaluated: Transformations are not executed until an action is called

4.2 Creating RDDs

```
from pyspark import SparkContext
sc = spark.sparkContext

# Method 1: parallelize a Python collection
rdd = sc.parallelize([1, 2, 3, 4, 5], numSlices=3)
# numSlices = number of partitions

# Method 2: read from a text file
rdd = sc.textFile("hdfs:///data/logs/*.txt")
# Each line becomes one element in the RDD

# Method 3: from a DataFrame
rdd = df.rdd  # converts DataFrame to RDD of Row objects

# Method 4: read whole text files
rdd = sc.wholeTextFiles("hdfs:///data/json/")
# Returns (filename, content) pairs
```

4.3 RDD Transformations (Lazy)

```
# map: apply function to each element
squared = rdd.map(lambda x: x * x)
# [1,2,3,4,5] --> [1,4,9,16,25]

# filter: keep elements matching condition
evens = rdd.filter(lambda x: x % 2 == 0)
# [1,2,3,4,5] --> [2,4]

# flatMap: map then flatten
words = lines_rdd.flatMap(lambda line: line.split(" "))
# ["hello world","foo bar"] --> ["hello","world","foo","bar"]

# reduceByKey: group by key and reduce values
word_counts = words.map(lambda w: (w, 1)).reduceByKey(lambda a, b: a + b)
# [("hello",1),("world",1),("hello",1)] --> [("hello",2),("world",1)]

# groupByKey: group values by key (avoid - causes shuffle + memory issues)
grouped = pairs_rdd.groupByKey()

# sortByKey: sort by key
sorted_rdd = pairs_rdd.sortByKey(ascending=True)

# join: join two RDDs by key
joined = rdd_a.join(rdd_b)  # inner join
# rdd_a: [(1,"Alice"),(2,"Bob")]
# rdd_b: [(1,"NYC"),(2,"LA")]
# result: [(1,("Alice","NYC")),(2,("Bob","LA"))]

# distinct: remove duplicates
unique = rdd.distinct()

# union: combine two RDDs
combined = rdd_a.union(rdd_b)
```

4.4 RDD Actions (Trigger Execution)

```
# collect: return all elements to driver (dangerous for large RDDs!)
result = rdd.collect()  # [1, 2, 3, 4, 5]

# count: number of elements
n = rdd.count()  # 5

# take: return first N elements
first_three = rdd.take(3)  # [1, 2, 3]

# first: return first element
one = rdd.first()  # 1

# reduce: aggregate all elements
total = rdd.reduce(lambda a, b: a + b)  # 15

# saveAsTextFile: write to file system
rdd.saveAsTextFile("hdfs:///output/results/")

# foreach: apply function to each element (no return)
rdd.foreach(lambda x: print(x))

# countByKey: count elements per key
counts = pairs_rdd.countByKey()  # {"hello": 2, "world": 1}
```

4.5 Lazy Evaluation

Spark does NOT execute transformations immediately. Instead, it builds a DAG of transformations and only executes when an action is called. This allows Spark to optimize the execution plan.

```
# These lines do NOT execute anything yet:
rdd = sc.textFile("big_file.txt")         # nothing happens
words = rdd.flatMap(lambda l: l.split())  # nothing happens
filtered = words.filter(lambda w: len(w) > 5)  # nothing happens
mapped = filtered.map(lambda w: (w, 1))   # nothing happens

# THIS triggers execution of the entire chain:
result = mapped.reduceByKey(lambda a, b: a + b).collect()
# NOW Spark reads the file, splits, filters, maps, reduces, collects
```

4.6 When to Use RDD vs DataFrame

Short answer: almost NEVER use RDDs in modern Spark. Use DataFrames.

Aspect	RDD	DataFrame
Performance	No Catalyst optimizer, no Tungsten	Catalyst optimizer + Tungsten = 10-100x faster
API	Low-level functional (map/filter/reduce)	High-level declarative (select/filter/groupBy)
Type Safety	Yes (in Scala)	Partial (Column type checked at runtime)
Schema	No schema	Has schema (StructType)
SQL Support	No	Yes (spark.sql)
Use When	Custom partitioning, low-level control	99% of data engineering tasks

Chapter 5: Partitions & Parallelism

5.1 What Are Partitions?

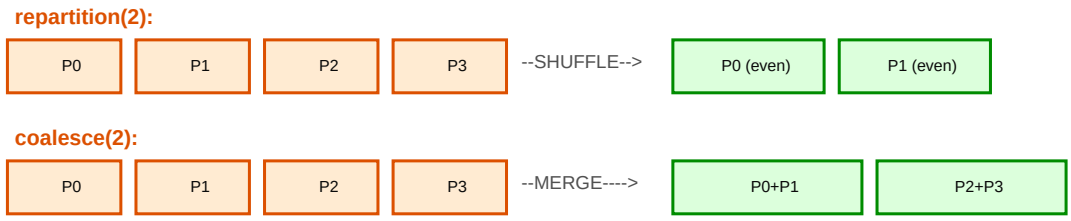
A partition is a chunk of data that is processed by a single task on a single executor. Partitions are the fundamental unit of parallelism in Spark. More partitions = more parallelism, but too many = overhead.

- Each partition is processed by exactly one task
- Tasks run in parallel across executors
- The number of partitions determines the degree of parallelism
- Default partition count when reading files: depends on file size and block size
- Default shuffle partition count: spark.sql.shuffle.partitions = 200

5.2 repartition() vs coalesce()

Both change the number of partitions, but they work very differently:

Aspect	repartition(N)	coalesce(N)
Direction	Increase OR decrease partitions	Only DECREASE partitions
Shuffle	ALWAYS triggers a full shuffle	NO shuffle (combines existing partitions)
Data Distribution	Evenly distributed across partitions	Uneven (merges adjacent partitions)
Use Case	Need even distribution (before join/groupBy)	Reduce output file count before write
Performance	Slower (full shuffle)	Faster (no data movement across network)



5.3 Partition Skew

Partition skew occurs when one partition has much more data than others. This causes one task to take much longer, making the entire stage wait for it.

- Symptom: In Spark UI, one task takes 10x-100x longer than others in the same stage
- Common cause: groupBy/join on a column with very uneven value distribution (e.g., country="US" has 80% of data)
- Fix 1: Salting - add a random number to the skewed key, process, then aggregate
- Fix 2: Broadcast join - if one side is small, broadcast it to avoid shuffle entirely
- Fix 3: AQE (Adaptive Query Execution) can automatically detect and fix skew in Spark 3.0+

5.4 Code: Checking and Changing Partition Count

```
# Check partition count
df = spark.read.parquet("s3://data/orders/")
print(f"Partitions: {df.rdd.getNumPartitions()}")  # e.g., 200

# Repartition (with shuffle) - for even distribution
df = df.repartition(100)  # 100 even partitions
df = df.repartition(100, "customer_id")  # hash partition by column

# Coalesce (without shuffle) - to reduce file count before write
df.coalesce(10).write.parquet("s3://output/orders/")

# Set default shuffle partitions
spark.conf.set("spark.sql.shuffle.partitions", "100")

# Check partition sizes (for detecting skew)
from pyspark.sql.functions import spark_partition_id, count
df.groupBy(spark_partition_id().alias("partition_id")) \
  .agg(count("*").alias("row_count")) \
  .orderBy("row_count", ascending=False) \
  .show(10)
```

PART 2: SPARK SQL & DATAFRAMES

Chapter 6: DataFrame Basics

DataFrames are the primary data abstraction in modern Spark. A DataFrame is a distributed collection of data organized into named columns - like a table in a database or a Pandas DataFrame, but distributed across a cluster.

6.1 Creating DataFrames

```
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import *

spark = SparkSession.builder.appName("DataFrameBasics").getOrCreate()

# Method 1: From CSV
df = spark.read.csv("data/orders.csv", header=True, inferSchema=True)

# Method 2: From JSON
df = spark.read.json("data/orders.json")

# Method 3: From Parquet (preferred format)
df = spark.read.parquet("data/orders.parquet")

# Method 4: From a Python list
data = [
    (1, "Alice", 29, "NYC"),
    (2, "Bob", 35, "LA"),
    (3, "Charlie", 42, "Chicago")
]
columns = ["id", "name", "age", "city"]
df = spark.createDataFrame(data, columns)

# Method 5: From a Python list with explicit schema
schema = StructType([
    StructField("id", IntegerType(), nullable=False),
    StructField("name", StringType(), nullable=True),
    StructField("age", IntegerType(), nullable=True),
    StructField("city", StringType(), nullable=True)
])
df = spark.createDataFrame(data, schema)

# Method 6: From an RDD
rdd = sc.parallelize(data)
df = spark.createDataFrame(rdd, columns)
```

6.2 Schema Definition

```
from pyspark.sql.types import *

# Explicit schema definition (recommended for production)
order_schema = StructType([
    StructField("order_id", LongType(), nullable=False),
    StructField("customer_id", IntegerType(), nullable=False),
    StructField("order_date", DateType(), nullable=True),
    StructField("amount", DecimalType(14, 2), nullable=True),
    StructField("status", StringType(), nullable=True),
    # Nested struct
    StructField("address", StructType([
        StructField("street", StringType()),
        StructField("city", StringType()),
        StructField("state", StringType()),
        StructField("zip", StringType())
    ])),
    # Array type
    StructField("items", ArrayType(StructType([
        StructField("product_id", IntegerType()),
        StructField("quantity", IntegerType()),
        StructField("price", DecimalType(10, 2))
    ])))
])
```

```
# Read with explicit schema (faster than inferSchema)
df = spark.read.schema(order_schema).json("data/orders.json")

# DDL-style schema string (simpler for flat schemas)
schema_str = "order_id LONG, customer_id INT, order_date DATE, amount DECIMAL(14,2)"
df = spark.read.schema(schema_str).csv("data/orders.csv", header=True)
```

TIP: Always define schemas explicitly in production. Schema inference (inferSchema=True) reads the entire file to determine types, which is slow and can be wrong.

6.3 Inspecting DataFrames

```
# Show first 20 rows (formatted table)
df.show()          # default: 20 rows, truncate=True
df.show(5, False)  # 5 rows, no truncation

# Print schema
df.printSchema()
# root
#  |-- order_id: long (nullable = false)
#  |-- customer_id: integer (nullable = false)
#  |-- amount: decimal(14,2) (nullable = true)

# Column names and types
print(df.columns)  # ['order_id', 'customer_id', 'amount', ...]
print(df.dtypes)   # [('order_id', 'bigint'), ('customer_id', 'int'), ...]

# Statistical summary
df.describe().show()  # count, mean, stddev, min, max for numeric columns
df.summary().show()   # includes percentiles

# Row count
print(df.count())  # e.g., 1000000
```

6.4 Selecting Columns

```
# Method 1: column names as strings
df.select("name", "age", "city").show()

# Method 2: using col() function
from pyspark.sql.functions import col
df.select(col("name"), col("age"), col("city")).show()

# Method 3: using df["column"] syntax
df.select(df["name"], df["age"]).show()

# Method 4: using expr() for expressions
from pyspark.sql.functions import expr
df.select(
    expr("name"),
    expr("age + 10 AS age_plus_10"),
    expr("UPPER(city) AS city_upper")
).show()

# Select all columns
df.select("*").show()

# Select with alias
df.select(
    col("name").alias("customer_name"),
    col("age").alias("customer_age")
).show()
```

Chapter 7: DataFrame Transformations

Transformations create new DataFrames from existing ones. They are lazy - Spark does not execute them until an action is called.

7.1 filter() / where()

```
# filter and where are exactly the same
df.filter(col("age") > 30).show()
df.where(col("age") > 30).show()

# Multiple conditions (use & for AND, | for OR, ~ for NOT)
df.filter(
    (col("age") > 25) & (col("city") == "NYC")
).show()

df.filter(
    (col("status") == "active") | (col("status") == "pending")
).show()

# String conditions (SQL-like)
df.filter("age > 30 AND city = 'NYC'").show()

# IN clause
df.filter(col("city").isin("NYC", "LA", "Chicago")).show()

# LIKE pattern matching
df.filter(col("name").like("A%")).show()     # starts with A
df.filter(col("name").rlike("^[A-C]")).show() # regex

# BETWEEN
df.filter(col("age").between(25, 35)).show()

# IS NULL / IS NOT NULL
df.filter(col("email").isNull()).show()
df.filter(col("email").isNotNull()).show()
```

7.2 withColumn()

```
# Add a new column
df = df.withColumn("age_group",
    when(col("age") < 30, "young")
    .when(col("age") < 50, "middle")
    .otherwise("senior")
)

# Modify existing column
df = df.withColumn("name", upper(col("name")))

# Add constant column
df = df.withColumn("country", lit("US"))

# Add computed column
df = df.withColumn("tax", col("amount") * 0.08)

# Add multiple columns at once (chaining)
df = df \
    .withColumn("full_name", concat(col("first"), lit(" "), col("last"))) \
    .withColumn("year", year(col("order_date"))) \
    .withColumn("is_large", col("amount") > 1000)
```

7.3 Other Common Transformations

```
# withColumnRenamed
df = df.withColumnRenamed("old_name", "new_name")

# drop columns
df = df.drop("temp_col", "debug_col")

# distinct (all columns)
df_unique = df.distinct()

# dropDuplicates (on specific columns)
df_dedup = df.dropDuplicates(["customer_id", "order_date"])
```

```
# sort / orderBy (same function)
df.sort(col("amount").desc()).show()
df.orderBy("name", ascending=True).show()
df.orderBy(col("date").desc(), col("amount").asc()).show()

# limit
df.limit(100).show()

# union (same schema required)
df_combined = df_2024.union(df_2025)

# unionByName (matches by column name, not position)
df_combined = df_a.unionByName(df_b, allowMissingColumns=True)

# sample
df_sample = df.sample(fraction=0.1, seed=42)  # 10% sample
```

Chapter 8: Aggregations & GroupBy

8.1 groupBy() with agg()

```
from pyspark.sql.functions import (
    count, sum, avg, min, max,
    collect_list, collect_set, countDistinct
)

# Simple groupBy
df.groupBy("city").count().show()
df.groupBy("city").agg(count("*").alias("total")).show()

# Multiple aggregations
df.groupBy("city").agg(
    count("*").alias("order_count"),
    sum("amount").alias("total_revenue"),
    avg("amount").alias("avg_order_value"),
    min("amount").alias("min_order"),
    max("amount").alias("max_order"),
    countDistinct("customer_id").alias("unique_customers")
).show()

# GroupBy multiple columns
df.groupBy("city", "status").agg(
    count("*").alias("count"),
    sum("amount").alias("total")
).orderBy("city", "status").show()

# Collect values into a list/set
df.groupBy("customer_id").agg(
    collect_list("product").alias("all_products"),
    collect_set("category").alias("unique_categories")
).show()
```

8.2 pivot()

```
# Pivot: rows to columns (cross-tabulation)
# Before: city | quarter | revenue
# After:  city | Q1 | Q2 | Q3 | Q4

df.groupBy("city") \
  .pivot("quarter", ["Q1", "Q2", "Q3", "Q4"]) \
  .agg(sum("revenue")) \
  .show()

# Always specify pivot values for better performance!
# Without values, Spark does an extra pass to find distinct values
```

8.3 rollup() and cube()

```
# rollup: hierarchical sub-totals
# Adds sub-totals for each level of the grouping hierarchy
df.rollup("region", "city").agg(
    sum("revenue").alias("total_revenue")
).orderBy("region", "city").show()
# Output includes:
#   region=East, city=NYC   --> revenue for East/NYC
#   region=East, city=null  --> subtotal for East
#   region=null, city=null  --> grand total

# cube: all combinations of sub-totals
df.cube("region", "city").agg(
    sum("revenue").alias("total_revenue")
).show()
# Includes: East/NYC, East/null, null/NYC, null/null (all combos)
```

Chapter 9: Joins

Joins combine rows from two DataFrames based on a matching condition. Understanding join types and their performance implications is critical for data engineering.

9.1 Join Types

Join Type	Keeps From Left	Keeps From Right	Description
inner	Matching only	Matching only	Only rows that match in both DataFrames
left (left_outer)	All rows	Matching only	All left rows + matching right (null if no match)
right (right_outer)	Matching only	All rows	All right rows + matching left (null if no match)
full (full_outer)	All rows	All rows	All rows from both (null where no match)
cross	All rows	All rows	Cartesian product (every row x every row)
left_semi	Matching only	Nothing	Left rows that have a match in right (like IN)
left_anti	Non-matching only	Nothing	Left rows with NO match in right (like NOT IN)

9.2 Join Syntax and Examples

```
orders = spark.table("orders")      # order_id, customer_id, amount
customers = spark.table("customers") # customer_id, name, city

# INNER JOIN
result = orders.join(customers,
    orders.customer_id == customers.customer_id,
    "inner"
)

# LEFT JOIN
result = orders.join(customers,
    orders.customer_id == customers.customer_id,
    "left"
)

# RIGHT JOIN
result = orders.join(customers,
    orders.customer_id == customers.customer_id,
    "right"
)

# FULL OUTER JOIN
result = orders.join(customers,
    orders.customer_id == customers.customer_id,
    "full"
)

# CROSS JOIN (use with extreme caution!)
result = orders.crossJoin(customers)

# LEFT SEMI JOIN (like WHERE customer_id IN (...))
result = orders.join(customers,
    orders.customer_id == customers.customer_id,
    "left_semi"
)

# LEFT ANTI JOIN (like WHERE customer_id NOT IN (...))
result = orders.join(customers,
    orders.customer_id == customers.customer_id,
    "left_anti"
)

# Join on multiple columns
result = df1.join(df2,
    (df1.col_a == df2.col_a) & (df1.col_b == df2.col_b),
    "inner"
)

# Shorthand: when column names are the same
result = orders.join(customers, "customer_id", "inner")
result = orders.join(customers, ["customer_id", "region"], "inner")
```

9.3 Broadcast Join

```
from pyspark.sql.functions import broadcast

# Broadcast the SMALL DataFrame to all executors
# Avoids shuffle of the large DataFrame
result = large_df.join(
    broadcast(small_df),  # small_df sent to every executor
    "join_key",
    "inner"
)

# Auto-broadcast threshold (default: 10MB)
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "50m")

# Disable auto-broadcast (force sort-merge join)
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")
```

TIP: Always broadcast the smaller table in a join. If the small table fits in memory (< 1GB), broadcast join can be 10x-100x faster than sort-merge join because it eliminates the shuffle of the large table.

9.4 Handling Duplicate Column Names

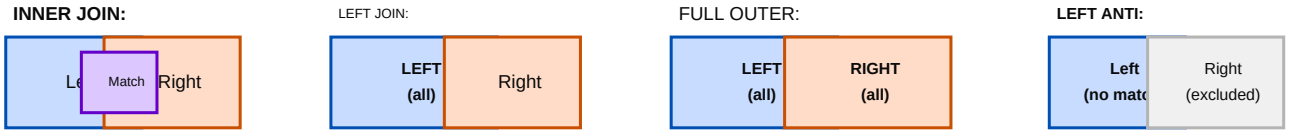
```
# Problem: after join, both DataFrames have "customer_id"
result = orders.join(customers, orders.customer_id == customers.customer_id)
# result.select("customer_id")  --> ERROR: ambiguous reference

# Solution 1: Use shorthand join (auto-deduplicates)
result = orders.join(customers, "customer_id")

# Solution 2: Alias DataFrames
o = orders.alias("o")
c = customers.alias("c")
result = o.join(c, col("o.customer_id") == col("c.customer_id"))
result.select(col("o.customer_id"), col("c.name")).show()

# Solution 3: Drop duplicate column after join
result = orders.join(customers, orders.customer_id == customers.customer_id)
result = result.drop(customers.customer_id)
```

9.5 Join Types Visual



Chapter 10: Window Functions

Window functions perform calculations across a set of rows related to the current row, without collapsing rows (unlike groupBy). They are essential for ranking, running totals, and row-level comparisons.

10.1 Window Specification

```
from pyspark.sql.window import Window
from pyspark.sql.functions import (
    row_number, rank, dense_rank,
    lag, lead, sum, avg,
    first, last, ntile
)

# Basic window: partition by department, order by salary desc
w = Window.partitionBy("department").orderBy(col("salary").desc())

# Window with frame: for running totals
w_running = Window.partitionBy("department") \
    .orderBy("order_date") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

# Window for range-based calculations
w_range = Window.partitionBy("department") \
    .orderBy("salary") \
    .rangeBetween(-1000, 1000)  # salary +/- 1000
```

10.2 ROW_NUMBER, RANK, DENSE_RANK

```
w = Window.partitionBy("department").orderBy(col("salary").desc())

df = df.withColumn("row_num", row_number().over(w))
df = df.withColumn("rank", rank().over(w))
df = df.withColumn("dense_rank", dense_rank().over(w))

# Example result:
# dept   | name    | salary | row_num | rank | dense_rank
# Sales  | Alice   | 90000  | 1       | 1    | 1
# Sales  | Bob     | 85000  | 2       | 2    | 2
# Sales  | Charlie | 85000  | 3       | 2    | 2
# Sales  | Dave    | 80000  | 4       | 4    | 3  <-- rank skips, dense_rank does not

# Common pattern: get top N per group
top_3 = df.withColumn("rn", row_number().over(w)) \
          .filter(col("rn") <= 3) \
          .drop("rn")
```

10.3 LAG and LEAD

```
w = Window.partitionBy("customer_id").orderBy("order_date")

# LAG: previous row value
df = df.withColumn("prev_amount", lag("amount", 1).over(w))
df = df.withColumn("prev_date", lag("order_date", 1).over(w))

# LEAD: next row value
df = df.withColumn("next_amount", lead("amount", 1).over(w))

# Calculate change from previous order
df = df.withColumn("amount_change",
    col("amount") - lag("amount", 1).over(w)
)

# Days since last order
df = df.withColumn("days_since_last",
    datediff(col("order_date"), lag("order_date", 1).over(w))
)
```

10.4 Running Totals and Moving Averages

```
# Running total
w_running = Window.partitionBy("customer_id") \
    .orderBy("order_date") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

df = df.withColumn("cumulative_total", sum("amount").over(w_running))

# Moving average (last 3 orders)
w_moving = Window.partitionBy("customer_id") \
    .orderBy("order_date") \
    .rowsBetween(-2, Window.currentRow)

df = df.withColumn("moving_avg_3", avg("amount").over(w_moving))
```

10.5 FIRST_VALUE, LAST_VALUE, NTILE

```
w = Window.partitionBy("department").orderBy("salary")

# First and last values in window
df = df.withColumn("lowest_paid", first("name").over(w))
df = df.withColumn("highest_paid", last("name").over(
    w.rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)
))

# NTILE: divide into N buckets
df = df.withColumn("salary_quartile", ntile(4).over(w))
# Assigns each row to bucket 1,2,3,4 based on salary within dept
```

Chapter 11: Working with Different Data Types

11.1 String Functions

```
from pyspark.sql.functions import (
    concat, concat_ws, substring, trim, ltrim, rtrim,
    lower, upper, initcap, length,
    regexp_replace, regexp_extract, split,
    lpad, rpad, instr, locate
)

df = df.withColumn("full_name", concat(col("first"), lit(" "), col("last")))
df = df.withColumn("csv_parts", concat_ws(",", col("a"), col("b"), col("c")))
df = df.withColumn("area_code", substring(col("phone"), 1, 3))
df = df.withColumn("cleaned", trim(col("name")))
df = df.withColumn("lower_name", lower(col("name")))
df = df.withColumn("upper_name", upper(col("name")))
df = df.withColumn("title_name", initcap(col("name")))
df = df.withColumn("name_len", length(col("name")))

# Regex
df = df.withColumn("clean_phone",
    regexp_replace(col("phone"), "[^0-9]", "")  # remove non-digits
)
df = df.withColumn("domain",
    regexp_extract(col("email"), "@(.+)", 1)  # extract domain
)

# Split string into array
df = df.withColumn("tags_array", split(col("tags"), ","))
```

11.2 Date/Time Functions

```
from pyspark.sql.functions import (
    current_date, current_timestamp,
    date_add, date_sub, datediff, months_between,
    date_format, to_date, to_timestamp,
    year, month, dayofmonth, dayofweek, hour, minute,
    date_trunc, last_day, next_day
)

df = df.withColumn("today", current_date())
df = df.withColumn("now", current_timestamp())
df = df.withColumn("next_week", date_add(col("order_date"), 7))
df = df.withColumn("days_ago", datediff(current_date(), col("order_date")))
df = df.withColumn("formatted", date_format(col("order_date"), "yyyy-MM-dd"))
df = df.withColumn("parsed", to_date(col("date_str"), "MM/dd/yyyy"))
df = df.withColumn("yr", year(col("order_date")))
df = df.withColumn("mo", month(col("order_date")))
df = df.withColumn("day", dayofmonth(col("order_date")))
df = df.withColumn("month_start", date_trunc("month", col("order_date")))
```

11.3 Null Handling

```
from pyspark.sql.functions import coalesce, when, isnull

# Check for nulls
df.filter(col("email").isNull()).show()
df.filter(col("email").isNotNull()).show()

# coalesce: return first non-null value
df = df.withColumn("contact",
    coalesce(col("email"), col("phone"), lit("unknown"))
)

# Fill nulls with specific values
df = df.na.fill(0, ["amount", "quantity"])  # numeric columns
df = df.na.fill("N/A", ["status", "city"]) # string columns
df = df.na.fill({"amount": 0, "status": "unknown"})  # dict form

# Drop rows with nulls
df_clean = df.na.drop()             # any column has null
df_clean = df.na.drop("all")        # all columns are null
df_clean = df.na.drop(subset=["email", "name"])  # specific columns
```

11.4 Type Casting and Array/Map Functions

```
# Type casting
df = df.withColumn("amount_int", col("amount").cast("integer"))
df = df.withColumn("amount_str", col("amount").cast(StringType()))

# Array functions
from pyspark.sql.functions import (
    explode, explode_outer, posexplode,
    array_contains, size, array, array_distinct, sort_array
)
df.select(explode(col("items")).alias("item")).show()  # one row per element
df = df.withColumn("has_phone", array_contains(col("devices"), "phone"))
df = df.withColumn("num_items", size(col("items")))

# Map functions
from pyspark.sql.functions import (
    map_keys, map_values, map_from_arrays, create_map
)
df = df.withColumn("keys", map_keys(col("properties")))
df = df.withColumn("vals", map_values(col("properties")))

# Create map from columns
df = df.withColumn("name_age_map",
    create_map(lit("name"), col("name"), lit("age"), col("age"))
)
```

Chapter 12: Spark SQL

Spark SQL lets you write standard SQL queries against DataFrames. Many data engineers prefer SQL for its readability and familiarity.

12.1 Creating Temp Views

```
# Register DataFrame as a temporary view
df.createOrReplaceTempView("orders")
# Available only in this SparkSession

# Global temp view (available across sessions)
df.createOrReplaceGlobalTempView("orders")
# Access as: global_temp.orders

# Drop temp view
spark.catalog.dropTempView("orders")
```

12.2 Running SQL Queries

```
# Basic SQL query
result = spark.sql("""
    SELECT customer_id,
           COUNT(*) AS order_count,
           SUM(amount) AS total_revenue,
           AVG(amount) AS avg_order
    FROM orders
    WHERE status = 'completed'
      AND order_date >= '2025-01-01'
    GROUP BY customer_id
    HAVING COUNT(*) > 5
    ORDER BY total_revenue DESC
    LIMIT 100
""")
result.show()

# SQL with subquery
result = spark.sql("""
    SELECT o.*, c.name, c.city
    FROM orders o
    INNER JOIN customers c ON o.customer_id = c.customer_id
    WHERE o.amount > (SELECT AVG(amount) FROM orders)
""")

# Window functions in SQL
result = spark.sql("""
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) as rn,
        SUM(salary) OVER (PARTITION BY dept) as dept_total,
        LAG(salary,1) OVER (PARTITION BY dept ORDER BY hire_date) as prev_salary
    FROM employees
""")
```

12.3 Registering UDFs for SQL

```
# Register a Python UDF for use in SQL
def classify_amount(amount):
    if amount is None:
        return "unknown"
    elif amount > 1000:
        return "high"
    elif amount > 100:
        return "medium"
    else:
        return "low"

spark.udf.register("classify_amount", classify_amount, StringType())

# Now use in SQL
result = spark.sql("""
    SELECT order_id, amount, classify_amount(amount) AS category
    FROM orders
""")
```

```
# DataFrame UDF (for use with withColumn)
from pyspark.sql.functions import udf
classify_udf = udf(classify_amount, StringType())
df = df.withColumn("category", classify_udf(col("amount")))
```

12.4 SQL vs DataFrame API Comparison

Operation	SQL	DataFrame API
Select	SELECT name, age	df.select("name","age")
Filter	WHERE age > 30	df.filter(col("age")>30)
GroupBy	GROUP BY city	df.groupBy("city")
Aggregate	SUM(amount)	agg(sum("amount"))
Join	JOIN t2 ON t1.id=t2.id	df1.join(df2,"id")
Sort	ORDER BY name	df.orderBy("name")
Window	ROW_NUMBER() OVER(...)	row_number().over(w)
Limit	LIMIT 10	df.limit(10)

Both produce the same execution plan. SQL is often preferred for complex queries with CTEs and subqueries. DataFrame API is preferred for dynamic/programmatic transformations.

PART 3: READING & WRITING DATA

Chapter 13: File Formats

Choosing the right file format is one of the most impactful decisions in data engineering. Each format has trade-offs in compression, read speed, schema support, and compatibility.

13.1 File Format Comparison

Format	Type	Compression	Schema	Speed	Best For
CSV	Row	Poor	No schema	Slow	Data exchange, Excel compat
JSON	Row	Poor	Self-describing	Slow	APIs, semi-structured data
Parquet	Columnar	Excellent	Embedded	Very Fast	Analytics, data lake storage
ORC	Columnar	Excellent	Embedded	Very Fast	Hive ecosystem
Avro	Row	Good	Schema registry	Fast	Kafka, streaming, schema evolution
Delta	Columnar+	Excellent	Enforced	Very Fast	ACID lakehouse tables

13.2 CSV

```
# Read CSV with all common options
df = spark.read.csv("data/orders.csv",
    header=True,          # first row is header
    inferSchema=True,     # auto-detect types (slow)
    sep=",",              # delimiter
    quote='"',            # quote character
    escape="\\",        # escape character
    nullValue="NULL",     # string to treat as null
    dateFormat="yyyy-MM-dd",
    timestampFormat="yyyy-MM-dd HH:mm:ss",
    multiLine=True,       # handle multi-line values
    encoding="UTF-8"
)

# Write CSV
df.write.csv("output/orders.csv",
    header=True,
    mode="overwrite",
    sep=","
)
```

13.3 JSON

```
# Read JSON (single-line - default, one JSON object per line)
df = spark.read.json("data/orders.json")

# Multi-line JSON (entire file is one JSON array)
df = spark.read.json("data/orders.json", multiLine=True)

# Write JSON
df.write.json("output/orders.json", mode="overwrite")
```

13.4 Parquet (Preferred Format)

```
# Read Parquet
df = spark.read.parquet("data/orders.parquet")

# Write Parquet
df.write.parquet("output/orders.parquet",
    mode="overwrite",
    compression="snappy"  # snappy (default), gzip, lz4, zstd
)

# Key Parquet advantages:
# - Columnar: only reads columns you SELECT (huge for wide tables)
# - Predicate pushdown: filters applied at file level (skip entire files)
# - Compression: 5-10x smaller than CSV
```

```
# - Schema embedded: no need to specify schema on read
# - Type-safe: preserves data types exactly
```

13.5 Avro

```
# Read Avro (requires spark-avro package)
df = spark.read.format("avro").load("data/orders.avro")

# Write Avro
df.write.format("avro").save("output/orders.avro")

# Avro is best for:
# - Kafka messages (row-based, fast serialization)
# - Schema evolution (backward/forward compatible)
# - Row-based operations (not analytics)
```

Chapter 14: Reading Data

14.1 General Read Pattern

```
# General pattern
df = spark.read \
    .format("parquet") \
    .option("key", "value") \
    .schema(my_schema) \
    .load("s3://bucket/path/")

# Read from different storage systems
df = spark.read.parquet("s3://bucket/data/")      # AWS S3
df = spark.read.parquet("abfss://container@storage.dfs.core.windows.net/data/")  # Azure ADLS
df = spark.read.parquet("gs://bucket/data/")       # GCP GCS
df = spark.read.parquet("hdfs:///data/")            # HDFS
df = spark.read.parquet("file:///local/path/data/") # Local filesystem
```

14.2 Reading from JDBC (Databases)

```
# Read from PostgreSQL
df = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://host:5432/mydb") \
    .option("dbtable", "public.orders") \
    .option("user", "admin") \
    .option("password", "secret") \
    .option("driver", "org.postgresql.Driver") \
    .option("fetchsize", "10000") \
    .option("numPartitions", "10") \
    .option("partitionColumn", "order_id") \
    .option("lowerBound", "1") \
    .option("upperBound", "10000000") \
    .load()

# Read with a SQL query instead of full table
df = spark.read.format("jdbc") \
    .option("url", "jdbc:postgresql://host:5432/mydb") \
    .option("query", "SELECT * FROM orders WHERE year = 2026") \
    .option("user", "admin") \
    .option("password", "secret") \
    .load()
```

14.3 Partition Discovery and Schema Merging

```
# Partition discovery: Spark auto-detects partition columns from path
# Directory structure: s3://data/orders/year=2025/month=01/data.parquet
df = spark.read.parquet("s3://data/orders/")
# df automatically has "year" and "month" columns from path!

# Partition filtering (partition pruning)
df = spark.read.parquet("s3://data/orders/") \
    .filter("year = 2026 AND month = 7")
# Spark only reads the partition directory year=2026/month=7/

# Schema merging (when Parquet files have different schemas)
df = spark.read.option("mergeSchema", "true") \
    .parquet("s3://data/orders/")
```

14.4 Handling Corrupt Records

```
# Mode: PERMISSIVE (default) - sets corrupt fields to null
df = spark.read.csv("data.csv",
    header=True,
    mode="PERMISSIVE",
    columnNameOfCorruptRecord="_corrupt_record"
)
corrupt = df.filter(col("_corrupt_record").isNotNull())

# Mode: DROPMALFORMED - silently drop bad rows
df = spark.read.csv("data.csv", header=True, mode="DROPMALFORMED")

# Mode: FAILFAST - throw exception on first bad row
df = spark.read.csv("data.csv", header=True, mode="FAILFAST")
```

Chapter 15: Writing Data

15.1 Write Modes

Mode	Behavior
append	Add new data to existing directory (no overwrite)
overwrite	Delete existing data and write new data
ignore	If data exists, do nothing (like CREATE IF NOT EXISTS)
error / errorifexists	Throw exception if data already exists (default)

15.2 Output Partitioning

```
# partitionBy: create directory structure by column values
df.write \
    .mode("overwrite") \
    .partitionBy("year", "month") \
    .parquet("s3://output/orders/")
# Creates: s3://output/orders/year=2026/month=07/part-00000.parquet

# Dynamic partition overwrite (only overwrite written partitions)
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")
df.write \
    .mode("overwrite") \
    .partitionBy("year", "month") \
    .parquet("s3://output/orders/")
# Only overwrites partitions that appear in df, not ALL partitions!
```

15.3 Controlling Output File Count

```
# Problem: Spark writes one file per partition = potentially 200 small files

# Solution 1: coalesce before write (no shuffle)
df.coalesce(10).write.parquet("output/")  # 10 output files

# Solution 2: repartition before write (with shuffle, even distribution)
df.repartition(10).write.parquet("output/")

# Solution 3: repartition by partition columns
df.repartition("year", "month") \
  .write.partitionBy("year", "month") \
  .parquet("output/")

# Solution 4: maxRecordsPerFile
df.write.option("maxRecordsPerFile", 1000000) \
  .parquet("output/")  # each file has at most 1M rows
```

15.4 bucketBy()

```
# bucketBy: pre-sort data into hash buckets for faster joins
# Must be used with saveAsTable (not save)
df.write \
    .mode("overwrite") \
    .bucketBy(100, "customer_id") \
    .sortBy("customer_id") \
    .saveAsTable("orders_bucketed")

# Now joins on customer_id avoid shuffle!
orders = spark.table("orders_bucketed")
customers = spark.table("customers_bucketed")  # also bucketed by customer_id
result = orders.join(customers, "customer_id")  # no shuffle needed!
```

Chapter 16: Delta Lake Integration

Delta Lake adds ACID transactions, time travel, and data quality features on top of Parquet. This chapter covers the basics - see the Delta Lake Complete Guide for full coverage.

16.1 Reading and Writing Delta Tables

```
# Write as Delta
df.write.format("delta").mode("overwrite") \
    .save("s3://lake/delta/orders/")

# Read Delta
df = spark.read.format("delta").load("s3://lake/delta/orders/")

# SQL: create managed Delta table
spark.sql("""
    CREATE TABLE orders USING DELTA
    AS SELECT * FROM parquet.`s3://raw/orders/`
""")
```

16.2 MERGE (Upsert)

```
from delta.tables import DeltaTable

target = DeltaTable.forPath(spark, "s3://lake/delta/orders/")
updates = spark.read.parquet("s3://staging/orders_today/")

(target.alias("t")
  .merge(updates.alias("s"), "t.order_id = s.order_id")
  .whenMatchedUpdateAll()
  .whenNotMatchedInsertAll()
  .execute()
)
```

16.3 Time Travel

```
# Read previous version
df_v5 = spark.read.format("delta") \
    .option("versionAsOf", 5).load("s3://lake/delta/orders/")

# Read at a specific timestamp
df_old = spark.read.format("delta") \
    .option("timestampAsOf", "2026-07-01").load("s3://lake/delta/orders/")

# View history
DeltaTable.forPath(spark, "s3://lake/delta/orders/").history().show()
```

16.4 OPTIMIZE and VACUUM

```
# Compact small files
spark.sql("OPTIMIZE orders")
spark.sql("OPTIMIZE orders ZORDER BY (customer_id)")

# Remove old files (default retention: 7 days)
spark.sql("VACUUM orders RETAIN 168 HOURS")
```

PART 4: PERFORMANCE & OPTIMIZATION

Chapter 17: Understanding the Spark UI

The Spark UI is your primary debugging and optimization tool. Learning to read it is a critical skill for any Spark developer.

17.1 Jobs Tab

Each action (count(), show(), write()) creates one job. The Jobs tab shows:

- Job ID and description (the action that triggered it)
- Status: running, succeeded, failed
- Duration: total time from start to finish
- Stages: number of stages in the job
- Tasks: total tasks across all stages

TIP: If a job takes too long, click into it to see which stage is slow. Then click into the stage to see task-level metrics.

17.2 Stages Tab

Stages are created at shuffle boundaries. Each stage contains tasks that can run in parallel. Key metrics:

- Shuffle Read: data read from other stages (indicates expensive shuffle)
- Shuffle Write: data written for downstream stages
- Task Duration: look for skew (one task takes 10x longer = data skew)
- Input Size: data read from storage
- Output Size: data written to storage
- GC Time: if high, increase executor memory or reduce data per task

17.3 SQL Tab

The SQL tab shows the execution plan for DataFrame/SQL operations. It includes:

- Logical Plan: what the query wants to do (before optimization)
- Optimized Logical Plan: after Catalyst optimizer rewrites
- Physical Plan: actual execution steps (scans, joins, exchanges)
- Look for: BroadcastHashJoin (good) vs SortMergeJoin (slower)
- Look for: FileScan with pushed filters (predicate pushdown working)
- Look for: Exchange (= shuffle, potential bottleneck)

17.4 How to Identify Bottlenecks

Symptom	Root Cause	Where to Look
One stage takes 90% of time	Data skew or insufficient parallelism	Stages tab: task duration distribution
High shuffle read/write	Too many shuffles or large shuffle	Stages tab: shuffle bytes
High GC time	Executor running out of memory	Executors tab: GC time column
Tasks failing with OOM	Data does not fit in memory	Executors tab: failed tasks count
Many retried tasks	Speculative execution or node issues	Stages tab: task error messages

Chapter 18: Common Performance Problems & Fixes

Problem 1: Data Skew

Data skew occurs when one partition has significantly more data than others, causing one task to take much longer than the rest.

```
# DIAGNOSIS: Check partition sizes
df.groupBy(spark_partition_id().alias("pid")) \
  .agg(count("*").alias("rows")) \
  .orderBy(col("rows").desc()) \
  .show(5)
# If max rows >> avg rows, you have skew

# FIX 1: Salting (add random prefix to skewed key)
from pyspark.sql.functions import rand, concat, lit
num_salts = 10
# Salt the skewed key
df_salted = df.withColumn("salted_key",
    concat(col("customer_id"), lit("_"), (rand() * num_salts).cast("int"))
)
# Aggregate on salted key, then aggregate again to remove salt
result = df_salted.groupBy("salted_key").agg(sum("amount").alias("partial_sum"))
result = result.withColumn("customer_id",
    split(col("salted_key"), "_")[0].cast("int")
).groupBy("customer_id").agg(sum("partial_sum").alias("total"))

# FIX 2: Broadcast join (if one side is small)
result = large_df.join(broadcast(small_df), "key")

# FIX 3: Enable AQE skew join optimization (Spark 3.0+)
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Problem 2: Too Many Small Files

Writing to a partitioned table can create thousands of tiny files (one per task per partition), which slows down subsequent reads.

```
# FIX 1: Coalesce before writing
df.coalesce(10).write.mode("overwrite").parquet("output/")

# FIX 2: Repartition by partition columns
df.repartition("year", "month") \
  .write.partitionBy("year", "month") \
  .parquet("output/")

# FIX 3: Use maxRecordsPerFile
df.write.option("maxRecordsPerFile", 500000) \
  .mode("overwrite").parquet("output/")

# FIX 4: Delta Lake OPTIMIZE (post-write compaction)
spark.sql("OPTIMIZE my_table")
```

Problem 3: Shuffle Spill to Disk

When shuffle data exceeds executor memory, Spark spills to disk, which is extremely slow.

```
# FIX 1: Increase executor memory
# spark-submit --executor-memory 16g

# FIX 2: Reduce shuffle partitions (if data is small)
spark.conf.set("spark.sql.shuffle.partitions", "50")

# FIX 3: Use AQE to auto-tune shuffle partitions
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

Problem 4: Full Table Scan

```
# FIX 1: Use partition pruning
# GOOD: filter on partition column
df = spark.read.parquet("s3://data/orders/") \
    .filter("year = 2026 AND month = 7")

# FIX 2: Use columnar format (Parquet/ORC)
# Only reads columns in SELECT, skips the rest
```

```
# FIX 3: Use predicate pushdown
# Spark pushes filters down to the file reader level
# Works with Parquet/ORC - reads only matching row groups
```

Problem 5: OOM (Out of Memory)

```
# FIX 1: Increase driver/executor memory
# spark-submit --driver-memory 8g --executor-memory 16g

# FIX 2: Reduce broadcast threshold (avoid broadcasting large tables)
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")

# FIX 3: Avoid collect() on large DataFrames
# BAD:  result = df.collect()  # brings ALL data to driver
# GOOD: result = df.limit(100).collect()  # only 100 rows

# FIX 4: Increase off-heap memory for Spark
# spark.memory.offHeap.enabled=true
# spark.memory.offHeap.size=4g
```

Chapter 19: Caching & Persistence

19.1 cache() vs persist() vs checkpoint()

Method	Storage	Lineage	Use Case
cache()	MEMORY_AND_DISK	Preserved	Reused DataFrame in same job
persist(level)	Configurable	Preserved	Fine-grained storage control
checkpoint()	Disk (reliable)	Truncated	Break long lineage chains

19.2 Storage Levels

```
from pyspark import StorageLevel

df.persist(StorageLevel.MEMORY_ONLY)         # fastest, fails if no room
df.persist(StorageLevel.MEMORY_AND_DISK)     # spills to disk (= cache())
df.persist(StorageLevel.DISK_ONLY)           # disk only (for very large data)
df.persist(StorageLevel.MEMORY_ONLY_2)       # replicated to 2 nodes
df.persist(StorageLevel.OFF_HEAP)            # off-heap memory

# cache() is shorthand for persist(StorageLevel.MEMORY_AND_DISK)
df.cache()  # same as df.persist(StorageLevel.MEMORY_AND_DISK)
```

19.3 When to Cache

- DO cache: DataFrame used multiple times (e.g., write + count + show)
- DO cache: Expensive transformations (complex joins, heavy aggregations)
- DO cache: Iterative algorithms (ML training loops)
- DO NOT cache: DataFrame used only once
- DO NOT cache: DataFrame too large to fit in memory (spill = slow)
- DO NOT cache: Data read from fast storage (Parquet with predicate pushdown)

19.4 Code Example

```
# Cache a reused DataFrame
df = spark.read.parquet("s3://data/orders/") \
    .filter(col("year") == 2026) \
    .join(broadcast(dim_customers), "customer_id")

df.cache()  # mark for caching

# First action materializes the cache
print(f"Total orders: {df.count()}")  # triggers computation + caching

# Second action uses the cache (much faster)
df.groupBy("city").agg(sum("amount")).show()

# Third action also uses cache
df.write.parquet("s3://output/enriched_orders/")

# Free memory when done
df.unpersist()

# Checkpoint: useful for breaking long lineage
spark.sparkContext.setCheckpointDir("s3://checkpoints/")
df.checkpoint()  # saves to disk, truncates lineage
```

Chapter 20: Broadcast Variables & Accumulators

20.1 Broadcast Variables

Broadcast variables are read-only shared variables that Spark sends to every executor once, instead of sending with every task. Use them for small lookup tables.

```
# Without broadcast: country_map is sent with EVERY task (N times)
# With broadcast: country_map is sent to each executor ONCE

# Create broadcast variable
country_codes = {"US": "United States", "UK": "United Kingdom",
                 "DE": "Germany", "FR": "France"}
bc_codes = spark.sparkContext.broadcast(country_codes)

# Use in a UDF
@udf(StringType())
def expand_country(code):
    return bc_codes.value.get(code, "Unknown")

df = df.withColumn("country_name", expand_country(col("country_code")))

# Broadcast join (most common use)
from pyspark.sql.functions import broadcast
result = large_orders.join(
    broadcast(small_countries),  # send to all executors
    "country_code"
)
```

20.2 Accumulators

Accumulators are distributed counters/sums. Only the driver can read the value; tasks can only add to it. Useful for debugging and monitoring.

```
# Create accumulators
error_count = spark.sparkContext.accumulator(0)
null_count = spark.sparkContext.accumulator(0)

def process_row(row):
    global error_count, null_count
    if row.amount is None:
        null_count += 1
        return None
    if row.amount < 0:
        error_count += 1
        return None
    return row.amount * 1.08  # add tax

# After action completes:
print(f"Errors: {error_count.value}")
print(f"Nulls: {null_count.value}")

# WARNING: accumulators in transformations may be counted multiple times
# if a task is retried. Only use in actions (foreach) for exact counts.
```

Chapter 21: Configuration Tuning

21.1 Key Spark Configurations

Config	Default	Description
spark.executor.memory	1g	Memory per executor JVM
spark.executor.cores	1	CPU cores per executor
spark.executor.instances	2	Number of executors (static alloc)
spark.driver.memory	1g	Memory for the driver process
spark.sql.shuffle.partitions	200	Partitions after shuffle operations
spark.sql.autoBroadcastJoinThreshold	10m	Auto-broadcast tables smaller than this
spark.serializer	JavaSerializer	Use KryoSerializer for 10x speed
spark.sql.adaptive.enabled	true (3.2+)	Enable Adaptive Query Execution
spark.dynamicAllocation.enabled	false	Auto-scale executors based on load
spark.speculation	false	Re-run slow tasks on another node

21.2 Adaptive Query Execution (AQE)

AQE (Spark 3.0+) dynamically optimizes query plans at runtime based on actual data statistics. It is one of the most important features for automatic performance tuning.

- Auto-coalesce shuffle partitions: Reduces 200 partitions to actual needed count
- Convert sort-merge join to broadcast join: If runtime data size is small enough
- Optimize skew joins: Splits skewed partitions into smaller pieces
- Always enable AQE in production!

```
# Enable AQE (recommended for ALL Spark 3.0+ jobs)
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.localShuffleReader.enabled", "true")

# With AQE, you can set a high shuffle partition count
# and AQE will automatically coalesce small partitions
spark.conf.set("spark.sql.shuffle.partitions", "500")
# AQE will reduce this to actual needed count at runtime
```

21.3 Memory Configuration

```
# Executor memory breakdown:
# Total executor memory = spark.executor.memory + spark.executor.memoryOverhead
#
# Inside executor memory:
#   60% = Execution memory (joins, aggregations, shuffles)
#   40% = Storage memory (cache, broadcast variables)
#   (Unified memory model - can borrow between execution and storage)
#
# spark.executor.memoryOverhead (default: max(384MB, 10% of executor memory))
# Used for: Python workers (PySpark), non-JVM memory, network buffers

# Production sizing example:
# If each YARN node has 64GB RAM and 16 cores:
# --executor-memory 14g   (leave room for overhead + OS)
# --executor-cores 5      (sweet spot: 2-5 cores per executor)
# --num-executors 10      (or use dynamic allocation)
# --executor-memoryOverhead 2g
```

PART 5: REAL-WORLD PATTERNS

Chapter 22: ETL Pipeline Pattern

A well-structured Spark ETL job follows the Read --> Validate --> Transform --> Write pattern. Here is a production-quality example:

```
import sys
import logging
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# ---- SETUP ----
logging.basicConfig(level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s")
log = logging.getLogger("OrdersETL")

spark = SparkSession.builder \
    .appName("OrdersETL") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()

# ---- READ ----
log.info("Reading raw orders...")
order_schema = StructType([
    StructField("order_id", LongType(), False),
    StructField("customer_id", IntegerType(), False),
    StructField("order_date", StringType(), True),
    StructField("amount", DoubleType(), True),
    StructField("status", StringType(), True),
])

raw = spark.read.schema(order_schema) \
    .option("mode", "PERMISSIVE") \
    .option("columnNameOfCorruptRecord", "_corrupt") \
    .csv("s3://raw/orders/2026/07/07/", header=True)

log.info(f"Read {raw.count()} raw records")
```

```
# ---- VALIDATE ----
log.info("Validating data...")
valid = raw.filter(
    col("order_id").isNotNull() &
    col("customer_id").isNotNull() &
    (col("amount") > 0) &
    col("status").isin("pending","completed","cancelled")
)
invalid = raw.subtract(valid)
invalid_count = invalid.count()
if invalid_count > 0:
    log.warning(f"Found {invalid_count} invalid records")
    invalid.write.mode("append") \
        .parquet("s3://quarantine/orders/2026-07-07/")

# ---- TRANSFORM ----
log.info("Transforming data...")
transformed = valid \
    .withColumn("order_date", to_date(col("order_date"), "yyyy-MM-dd")) \
    .withColumn("year", year(col("order_date"))) \
    .withColumn("month", month(col("order_date"))) \
    .withColumn("tax", col("amount") * 0.08) \
    .withColumn("total", col("amount") + col("amount") * 0.08) \
    .withColumn("etl_timestamp", current_timestamp()) \
    .drop("_corrupt")

# ---- WRITE ----
log.info("Writing to Delta Lake...")
transformed.repartition("year", "month") \
    .write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
```

```
    .partitionBy("year", "month") \
    .save("s3://lake/delta/orders/")

log.info("ETL complete!")
spark.stop()
```

TIP: Always define schemas explicitly (no inferSchema). Always validate data before transforming. Always log counts at each stage for monitoring. Always handle errors gracefully.

Chapter 23: Incremental Processing

23.1 Full Load vs Incremental Load

Aspect	Full Load	Incremental Load
What it does	Reads and rewrites ALL data	Processes only NEW/CHANGED data
Speed	Slow for large datasets	Fast (processes small delta)
Resource Usage	High (processes everything)	Low (processes changes only)
Complexity	Simple (overwrite)	More complex (tracking changes)
Use Case	Small tables, first load	Large tables, daily/hourly updates

23.2 Watermark-Based Incremental

```
# Track high-water mark (max timestamp) from previous run
# Store in a control table or file

# Read the high-water mark from last successful run
last_processed = spark.read.text("s3://control/orders_hwm.txt") \
    .first()[0]  # e.g., "2026-07-06 23:59:59"

# Read only new records (incremental)
new_records = spark.read.parquet("s3://raw/orders/") \
    .filter(col("updated_at") > last_processed)

log.info(f"Processing {new_records.count()} new records since {last_processed}")

# Process and write
new_records.write.format("delta") \
    .mode("append") \
    .save("s3://lake/delta/orders/")

# Update high-water mark
new_hwm = new_records.agg(max("updated_at")).first()[0]
spark.createDataFrame([(str(new_hwm),)], ["hwm"]) \
    .write.mode("overwrite") \
    .text("s3://control/orders_hwm.txt")
```

23.3 Delta Lake MERGE for CDC

```
from delta.tables import DeltaTable

# Read incremental CDC data (inserts + updates + deletes)
cdc = spark.read.parquet("s3://cdc/orders/2026-07-07/")

target = DeltaTable.forPath(spark, "s3://lake/delta/orders/")

(target.alias("t")
  .merge(cdc.alias("s"), "t.order_id = s.order_id")
  .whenMatchedUpdate(
      condition="s.op_type = 'U'",  # update
      set={"amount": "s.amount", "status": "s.status",
           "updated_at": "current_timestamp()"}
  )
  .whenMatchedDelete(condition="s.op_type = 'D'")  # delete
  .whenNotMatchedInsert(
      condition="s.op_type = 'I'",  # insert
      values={"order_id": "s.order_id", "customer_id": "s.customer_id",
              "amount": "s.amount", "status": "s.status",
              "created_at": "current_timestamp()",
              "updated_at": "current_timestamp()"}
  )
  .execute()
)
```

23.4 Auto Loader (Databricks)

```
# Auto Loader: automatically processes new files as they arrive
# Tracks which files have been processed (no manual watermark needed)

df = spark.readStream \
    .format("cloudFiles") \
    .option("cloudFiles.format", "json") \
    .option("cloudFiles.schemaLocation", "s3://schema/orders/") \
    .load("s3://landing/orders/")

df.writeStream \
    .format("delta") \
    .option("checkpointLocation", "s3://checkpoints/orders/") \
    .outputMode("append") \
    .trigger(availableNow=True) \
    .start("s3://lake/delta/orders/")
```

Chapter 24: Structured Streaming Basics

Structured Streaming treats streaming data as an unbounded table that grows continuously. It uses the same DataFrame API as batch processing.

24.1 readStream vs read

```
# BATCH: read (finite data)
df = spark.read.parquet("s3://data/orders/")

# STREAMING: readStream (infinite/continuous data)
df = spark.readStream \
    .format("parquet") \
    .schema(order_schema) \
    .load("s3://landing/orders/")

# Same transformations work on both!
result = df.filter(col("amount") > 100) \
    .groupBy("city") \
    .agg(sum("amount").alias("total"))
```

24.2 writeStream with Output Modes

```
# APPEND mode: only new rows (default, no aggregations)
query = df.writeStream \
    .format("parquet") \
    .outputMode("append") \
    .option("checkpointLocation", "s3://checkpoints/orders/") \
    .start("s3://output/orders/")

# COMPLETE mode: output entire result table (for aggregations)
query = result.writeStream \
    .format("console") \
    .outputMode("complete") \
    .start()

# UPDATE mode: only changed rows since last trigger
query = result.writeStream \
    .format("delta") \
    .outputMode("update") \
    .option("checkpointLocation", "s3://checkpoints/agg/") \
    .start("s3://lake/delta/order_agg/")
```

24.3 Triggers

```
# processingTime: run micro-batches at fixed intervals
query = df.writeStream \
    .trigger(processingTime="30 seconds") \
    .start(...)

# once: process all available data, then stop (like batch)
query = df.writeStream \
    .trigger(once=True) \
    .start(...)

# availableNow: process all available data in multiple batches, then stop
# More efficient than once for large backlogs
query = df.writeStream \
    .trigger(availableNow=True) \
    .start(...)
```

24.4 Watermarks for Late Data

```
# Watermark: tell Spark how late data can arrive
# After the watermark, Spark drops late events
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "orders") \
    .load() \
    .select(from_json(col("value").cast("string"), schema).alias("data")) \
    .select("data.*") \
    .withWatermark("event_time", "10 minutes") \
    .groupBy(
```

```
        window("event_time", "5 minutes"),
        "city"
    ) \
    .agg(sum("amount").alias("total"))
```

24.5 Streaming from Kafka

```
# Read from Kafka topic
kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "broker1:9092,broker2:9092") \
    .option("subscribe", "orders") \
    .option("startingOffsets", "latest") \
    .option("maxOffsetsPerTrigger", 100000) \
    .load()

# Kafka DataFrame has: key, value, topic, partition, offset, timestamp
# Parse the value (usually JSON)
parsed = kafka_df \
    .select(col("value").cast("string").alias("json")) \
    .select(from_json("json", order_schema).alias("data")) \
    .select("data.*")

# Write to Delta Lake
query = parsed.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "s3://checkpoints/kafka_orders/") \
    .trigger(processingTime="1 minute") \
    .start("s3://lake/delta/orders/")

# Monitor the streaming query
query.status          # current status
query.recentProgress  # recent metrics
query.awaitTermination()  # block until query stops
```

Chapter 25: Testing Spark Applications

25.1 Unit Testing with pytest

```
# conftest.py - shared fixtures
import pytest
from pyspark.sql import SparkSession

@pytest.fixture(scope="session")
def spark():
    """Create a SparkSession for testing (once per test session)."""
    spark = SparkSession.builder \
        .master("local[2]") \
        .appName("UnitTests") \
        .config("spark.sql.shuffle.partitions", "2") \
        .config("spark.ui.enabled", "false") \
        .getOrCreate()
    yield spark
    spark.stop()
```

25.2 Creating Test DataFrames

```
# test_transformations.py
from pyspark.sql.types import *
from pyspark.sql.functions import *

def test_add_tax_column(spark):
    """Test that add_tax correctly computes 8% tax."""
    # ARRANGE: create test data
    data = [(1, 100.0), (2, 200.0), (3, 0.0)]
    df = spark.createDataFrame(data, ["id", "amount"])

    # ACT: apply transformation
    result = df.withColumn("tax", col("amount") * 0.08)

    # ASSERT: verify result
    rows = result.collect()
    assert rows[0]["tax"] == 8.0
    assert rows[1]["tax"] == 16.0
    assert rows[2]["tax"] == 0.0
```

25.3 Comparing DataFrames

```
# Method 1: Using assertDataFrameEqual (PySpark 3.5+)
from pyspark.testing import assertDataFrameEqual

def test_transform(spark):
    input_df = spark.createDataFrame([(1, "a"), (2, "b")], ["id", "val"])
    expected = spark.createDataFrame([(1, "A"), (2, "B")], ["id", "val"])

    result = input_df.withColumn("val", upper(col("val")))
    assertDataFrameEqual(result, expected)

# Method 2: Manual comparison (works in all versions)
def assert_df_equal(actual, expected):
    """Compare two DataFrames regardless of row order."""
    assert actual.schema == expected.schema, "Schema mismatch"
    assert actual.count() == expected.count(), "Row count mismatch"
    diff = actual.subtract(expected)
    assert diff.count() == 0, f"Unexpected rows: {diff.collect()}"
```

25.4 Mocking External Dependencies

```
from unittest.mock import patch, MagicMock

def test_etl_with_mock_read(spark):
    """Test ETL logic without real S3 access."""
    # Create fake input data
    fake_data = spark.createDataFrame([
        (1, 100.0, "completed"),
        (2, -50.0, "completed"),  # invalid: negative
        (3, 200.0, "pending"),
    ], ["order_id", "amount", "status"])
```

```
    # Test your transformation logic
    valid = fake_data.filter(col("amount") > 0)
    assert valid.count() == 2

    result = valid.withColumn("tax", col("amount") * 0.08)
    assert result.filter(col("order_id") == 1).first()["tax"] == 8.0

# Run tests: pytest tests/ -v --tb=short
```

QUICK REFERENCE

PySpark Cheat Sheet

Operation	Code
Create SparkSession	SparkSession.builder.appName("app").getOrCreate()
Read CSV	spark.read.csv("path", header=True, inferSchema=True)
Read Parquet	spark.read.parquet("path")
Read JSON	spark.read.json("path")
Read Delta	spark.read.format("delta").load("path")
Show data	df.show(20, truncate=False)
Print schema	df.printSchema()
Select columns	df.select("col1", "col2")
Filter rows	df.filter(col("age") > 30)
Add column	df.withColumn("new", col("a") + col("b"))
Rename column	df.withColumnRenamed("old", "new")
Drop column	df.drop("col1", "col2")
Distinct rows	df.distinct() or df.dropDuplicates(["col"])
Sort	df.orderBy(col("amount").desc())
GroupBy + Agg	df.groupBy("city").agg(sum("amt").alias("total"))
Join	df1.join(df2, "key", "inner")
Broadcast join	df1.join(broadcast(df2), "key")
Window function	row_number().over(Window.partitionBy("x").orderBy("y"))
Union	df1.unionByName(df2, allowMissingColumns=True)
Cache	df.cache() / df.persist() / df.unpersist()
Write Parquet	df.write.mode("overwrite").parquet("path")
Write Delta	df.write.format("delta").mode("overwrite").save("path")
Partition output	df.write.partitionBy("year","month").parquet(...)
Coalesce	df.coalesce(10).write.parquet(...)
Repartition	df.repartition(100, "col")
Count	df.count()
Collect	df.collect() (careful: brings all to driver)
Register temp view	df.createOrReplaceTempView("orders")
Run SQL	spark.sql("SELECT * FROM orders WHERE ...")
UDF	spark.udf.register("fn", my_func, StringType())
Check partitions	df.rdd.getNumPartitions()

Spark SQL Functions Cheat Sheet

Category	Functions
Aggregation	count, sum, avg, min, max, collect_list, collect_set, countDistinct
String	concat, substring, trim, lower, upper, regexp_replace, split, length
Date/Time	current_date, date_add, datediff, date_format, to_date, year, month, day
Null	coalesce, isNull, isNotNull, na.fill, na.drop, when/otherwise
Window	row_number, rank, dense_rank, lag, lead, ntile, first, last
Math	abs, ceil, floor, round, log, pow, sqrt, rand
Conditional	when/otherwise, if (SQL), case/when (SQL), coalesce
Array	explode, array_contains, size, array_distinct, sort_array
Type	cast, col.cast("int"), col.cast(StringType())
Misc	lit (literal), col (column ref), expr (SQL expression), monotonically_increasing_id

Configuration Cheat Sheet

Config	Recommended Value	Notes
spark.sql.adaptive.enabled	true	ALWAYS enable in Spark 3.0+
spark.sql.shuffle.partitions	Auto (AQE)	200 default, AQE auto-tunes
spark.executor.memory	4g-16g	Depends on workload and cluster
spark.executor.cores	2-5	More than 5 causes GC issues
spark.driver.memory	2g-8g	Increase if collect() is needed
spark.sql.autoBroadcastJoinThreshold	10m-100m	Broadcast small tables
spark.serializer	KryoSerializer	10x faster than Java serializer
spark.dynamicAllocation.enabled	true	Auto-scale executors
spark.sql.adaptive.skewJoin.enabled	true	Fix skew automatically
spark.sql.files.maxPartitionBytes	128m	Max partition size for file reads
spark.sql.parquet.compression.codec	snappy	Good compression + speed
spark.speculation	false	Enable if stragglers are common

End of Guide

Apache Spark - Complete Learning Guide (Basic to Medium)

Practice each chapter with real datasets.
Build ETL pipelines and measure their performance.
Master the Spark UI - it is your best debugging tool.